

Environment Setup — Docker Compose Configuration

```
docker-compose.yml
  Run All Services
1  services:
  Run Service
2    dbms-project:
3      image: postgres:17
4      container_name: dbms-project-db
5      restart: unless-stopped
6
7      environment:
8        POSTGRES_USER: postgres
9        POSTGRES_PASSWORD: ****
10
11
12      ports:
13        - "5433:5432"
14
15      volumes:
16        - postgres_data:/var/lib/postgresql/data
17
18  volumes:
19    postgres_data:
```

Starting the Container and Connecting via psql (PowerShell)

```
TERMINAL  ...  docker + - [ ] [ ] ... | [ ] [ ] X

PS C:\Users\Gaurav Gautam\Desktop\BSc.CSIT\4th_SEMESTE
R\DBMS> docker compose up -d

[+] up 1/1
  ✓ Container dbms-project-db Running 0.0
s
PS C:\Users\Gaurav Gautam\Desktop\BSc.CSIT\4th_SEMESTE
R\DBMS> docker ps

CONTAINER ID   IMAGE          COMMAND
CREATED        STATUS        PORTS
NAMES
3ebaec32790c   postgres:17   "docker-entrypoint.s..."
11 minutes ago Up 11 minutes  0.0.0.0:5433->5432/tcp
dbms-project-db
PS C:\Users\Gaurav Gautam\Desktop\BSc.CSIT\4th_SEMESTE
R\DBMS> docker exec -it dbms-project-db psql -U postgr
es -d project
psql (17.10 (Debian 17.10-1.pgdg13+1))
Type "help" for help.

project=#
```

Database Creation and Management (DDL)

Creating a new database with `CREATE DATABASE college_db;`

```
postgres=# CREATE DATABASE college_db;
CREATE DATABASE
postgres=#
```

Listing every database with `\l` — `college_db` now appears alongside `postgres` and `project`.

List of databases								
Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges
college_db	postgres	UTF8	libc	en_US.utf8	en_US.utf8			
postgres	postgres	UTF8	libc	en_US.utf8	en_US.utf8			
project	postgres	UTF8	libc	en_US.utf8	en_US.utf8			
template0	postgres	UTF8	libc	en_US.utf8	en_US.utf8			=c/postgres + postgres=CTc/postgres
template1	postgres	UTF8	libc	en_US.utf8	en_US.utf8			=c/postgres + postgres=CTc/postgres

(5 rows)

Switching the active connection between databases using `\c college_db` and `\c postgres`.

```
postgres=# \c college_db
You are now connected to database "college_db" as user "postgres".
college_db=# \c postgres
You are now connected to database "postgres" as user "postgres".
postgres=#
```

Dropping the test database with `DROP DATABASE college_db;`, confirmed by `\l`. All further work is carried out inside the `project` database.

```
postgres=# DROP DATABASE college_db;
DROP DATABASE
postgres=# \l
```

List of databases								
Name	Owner	Encoding	Locale Provider	Collate	Ctype	Locale	ICU Rules	Access privileges
postgres	postgres	UTF8	libc	en_US.utf8	en_US.utf8			
project	postgres	UTF8	libc	en_US.utf8	en_US.utf8			
template0	postgres	UTF8	libc	en_US.utf8	en_US.utf8			=c/postgres + postgres=CTc/postgres
template1	postgres	UTF8	libc	en_US.utf8	en_US.utf8			=c/postgres + postgres=CTc/postgres

(4 rows)

CREATE TABLE — Building the Schema (DDL)

Creating the `Courses` table, with `course_id` as a `SERIAL PRIMARY KEY`.

```
project=# CREATE TABLE Courses (
  course_id      SERIAL PRIMARY KEY,
  course_code    VARCHAR(10) UNIQUE,
  course_name    VARCHAR(100),
  duration_years INT,
  description     TEXT
);
CREATE TABLE
project=#
```

Creating the Teachers table.

```
project=# CREATE TABLE Teachers (  
    teacher_id SERIAL PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name  VARCHAR(50),  
    email      VARCHAR(100) UNIQUE,  
    phone      VARCHAR(15),  
    department VARCHAR(100),  
    hire_date  DATE  
);  
CREATE TABLE  
project=#
```

Creating the Students table, with course_id referencing Courses(course_id).

```
project=# CREATE TABLE Students (  
    student_id SERIAL PRIMARY KEY,  
    course_id  INT NOT NULL REFERENCES Courses(course_id),  
    first_name VARCHAR(50),  
    last_name  VARCHAR(50),  
    email      VARCHAR(100),  
    phone      VARCHAR(15),  
    enrollment_year INT,  
    symbol_number VARCHAR(20) UNIQUE  
);  
CREATE TABLE  
project=#
```

Creating the Semesters table, with course_id referencing Courses(course_id).

```
project=# CREATE TABLE Semesters (  
    semester_id SERIAL PRIMARY KEY,  
    course_id  INT NOT NULL REFERENCES Courses(course_id),  
    semester_number INT,  
    semester_name VARCHAR(50),  
    start_date  DATE,  
    end_date    DATE  
);  
CREATE TABLE  
project=#
```

Creating the Subjects table, with semester_id referencing Semesters(semester_id).

```
project=# CREATE TABLE Subjects (  
    subject_id SERIAL PRIMARY KEY,  
    semester_id INT NOT NULL REFERENCES Semesters(semester_id),  
    subject_code VARCHAR(20),  
    subject_name VARCHAR(150),  
    credit_hours INT,  
    total_classes INT  
);  
CREATE TABLE  
project=#
```

Creating the Teacher_Subjects junction table linking Teachers and Subjects — this resolves the M:N Teaches relationship.

```
project=# CREATE TABLE Teacher_Subjects (  
    ts_id          SERIAL PRIMARY KEY,  
    teacher_id     INT NOT NULL REFERENCES Teachers(teacher_id),  
    subject_id     INT NOT NULL REFERENCES Subjects(subject_id),  
    academic_year  VARCHAR(10),  
    assign_date    DATE  
);  
CREATE TABLE  
project=#
```

Creating the Enrollments junction table linking Students and Semesters — this resolves the M:N Enrolled_in relationship.

```
project=# CREATE TABLE Enrollments (  
    enrollment_id  SERIAL PRIMARY KEY,  
    student_id     INT NOT NULL REFERENCES Students(student_id),  
    semester_id    INT NOT NULL REFERENCES Semesters(semester_id),  
    enrollment_date DATE,  
    status         VARCHAR(20)  
);  
CREATE TABLE  
project=#
```

Creating the Class_Sessions table, referencing both Subjects and Teachers.

```
project=# CREATE TABLE Class_Sessions (  
    session_id     SERIAL PRIMARY KEY,  
    subject_id     INT NOT NULL REFERENCES Subjects(subject_id),  
    teacher_id     INT NOT NULL REFERENCES Teachers(teacher_id),  
    session_date   DATE,  
    start_time     TIME,  
    end_time       TIME,  
    remarks        TEXT  
);  
CREATE TABLE  
project=#
```

Creating the Attendances table, referencing both Class_Sessions and Students.

```
project=# CREATE TABLE Attendances (  
    attendance_id  SERIAL PRIMARY KEY,  
    session_id     INT NOT NULL REFERENCES Class_Sessions(session_id),  
    student_id     INT NOT NULL REFERENCES Students(student_id),  
    status         VARCHAR(2),  
    remarks        TEXT,  
    recorded_at    TIMESTAMP DEFAULT NOW()  
);  
CREATE TABLE  
project=#
```

Verifying Table Structures with \d

Structure of the Courses table — columns, data types, default values and the primary key / unique indexes.

Table "public.courses"				
Column	Type	Collation	Nullable	Default
course_id	integer		not null	nextval('courses_course_id_seq'::regclass)
course_code	character varying(10)			
course_name	character varying(100)			
duration_years	integer			
description	text			

Indexes:

- "courses_pkey" PRIMARY KEY, btree (course_id)
- "courses_course_code_key" UNIQUE CONSTRAINT, btree (course_code)

Referenced by:

- TABLE "semesters" CONSTRAINT "semesters_course_id_fkey" FOREIGN KEY (course_id) REFERENCES courses(course_id)
- TABLE "students" CONSTRAINT "students_course_id_fkey" FOREIGN KEY (course_id) REFERENCES courses(course_id)

Structure of the Teachers table, including the tables that reference it as a foreign key.

Table "public.teachers"				
Column	Type	Collation	Nullable	Default
teacher_id	integer		not null	nextval('teachers_teacher_id_seq'::regclass)
first_name	character varying(50)			
last_name	character varying(50)			
email	character varying(100)			
phone	character varying(15)			
department	character varying(100)			
hire_date	date			

Indexes:

- "teachers_pkey" PRIMARY KEY, btree (teacher_id)
- "teachers_email_key" UNIQUE CONSTRAINT, btree (email)

Referenced by:

- TABLE "class_sessions" CONSTRAINT "class_sessions_teacher_id_fkey" FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id)
- TABLE "teacher_subjects" CONSTRAINT "teacher_subjects_teacher_id_fkey" FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id)

Structure of the Students table, including its foreign key to Courses and the tables that reference it.

Table "public.students"				
Column	Type	Collation	Nullable	Default
student_id	integer		not null	nextval('students_student_id_seq'::regclass)
course_id	integer		not null	
first_name	character varying(50)			
last_name	character varying(50)			
email	character varying(100)			
phone	character varying(15)			
enrollment_year	integer			
symbol_number	character varying(20)			

Indexes:

- "students_pkey" PRIMARY KEY, btree (student_id)
- "students_symbol_number_key" UNIQUE CONSTRAINT, btree (symbol_number)

Foreign-key constraints:

- "students_course_id_fkey" FOREIGN KEY (course_id) REFERENCES courses(course_id)

Referenced by:

- TABLE "attendances" CONSTRAINT "attendances_student_id_fkey" FOREIGN KEY (student_id) REFERENCES students(student_id)
- TABLE "enrollments" CONSTRAINT "enrollments_student_id_fkey" FOREIGN KEY (student_id) REFERENCES students(student_id)

Structure of the Semesters table and the tables that reference it.

Table "public.semesters"				
Column	Type	Collation	Nullable	Default
-----+-----+-----+-----+-----				
semester_id	integer		not null	nextval('semesters_semester_id_seq'::regclass)
course_id	integer		not null	
semester_number	integer			
semester_name	character varying(50)			
start_date	date			
end_date	date			
Indexes:				
"semesters_pkey" PRIMARY KEY, btree (semester_id)				
Foreign-key constraints:				
"semesters_course_id_fkey" FOREIGN KEY (course_id) REFERENCES courses(course_id)				
Referenced by:				
TABLE "enrollments" CONSTRAINT "enrollments_semester_id_fkey" FOREIGN KEY (semester_id) REFERENCES semesters(semester_id)				
TABLE "subjects" CONSTRAINT "subjects_semester_id_fkey" FOREIGN KEY (semester_id) REFERENCES semesters(semester_id)				

Structure of the Subjects table and the tables that reference it.

```
project=# \d Subjects
```

Table "public.subjects"				
Column	Type	Collation	Nullable	Default
subject_id	integer		not null	nextval('subjects_subject_id_seq'::regclass)
semester_id	integer		not null	
subject_code	character varying(20)			
subject_name	character varying(150)			
credit_hours	integer			
total_classes	integer			

Indexes:

"subjects_pkey" PRIMARY KEY, btree (subject_id)

Foreign-key constraints:

"subjects_semester_id_fkey" FOREIGN KEY (semester_id) REFERENCES semesters(semester_id)

Referenced by:

TABLE "class_sessions" CONSTRAINT "class_sessions_subject_id_fkey" FOREIGN KEY (subject_id) REFERENCES subjects(subject_id)

TABLE "teacher_subjects" CONSTRAINT "teacher_subjects_subject_id_fkey" FOREIGN KEY (subject_id) REFERENCES subjects(subject_id)

```
project=#
```

Structure of the Teacher_Subjects junction table.

```
project=# \d Teacher_Subjects
```

Table "public.teacher_subjects"				
Column	Type	Collation	Nullable	Default
ts_id	integer		not null	nextval('teacher_subjects_ts_id_seq'::regclass)
teacher_id	integer		not null	
subject_id	integer		not null	
academic_year	character varying(10)			
assign_date	date			

Indexes:

- "teacher_subjects_pkey" PRIMARY KEY, btree (ts_id)

Foreign-key constraints:

- "teacher_subjects_subject_id_fkey" FOREIGN KEY (subject_id) REFERENCES subjects(subject_id)
- "teacher_subjects_teacher_id_fkey" FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id)

```
project=#
```

Structure of the Enrollments junction table.

Table "public.enrollments"				
Column	Type	Collation	Nullable	Default
enrollment_id	integer		not null	nextval('enrollments_enrollment_id_seq'::regclass)
student_id	integer		not null	
semester_id	integer		not null	
enrollment_date	date			
status	character varying(20)			

Indexes:
 "enrollments_pkey" PRIMARY KEY, btree (enrollment_id)

Foreign-key constraints:
 "enrollments_semester_id_fkey" FOREIGN KEY (semester_id) REFERENCES semesters(semester_id)
 "enrollments_student_id_fkey" FOREIGN KEY (student_id) REFERENCES students(student_id)

Structure of the Class_Sessions table.

Table "public.class_sessions"				
Column	Type	Collation	Nullable	Default
session_id	integer		not null	nextval('class_sessions_session_id_seq'::regclass)
subject_id	integer		not null	
teacher_id	integer		not null	
session_date	date			
start_time	time without time zone			
end_time	time without time zone			
remarks	text			

Indexes:
 "class_sessions_pkey" PRIMARY KEY, btree (session_id)

Foreign-key constraints:
 "class_sessions_subject_id_fkey" FOREIGN KEY (subject_id) REFERENCES subjects(subject_id)
 "class_sessions_teacher_id_fkey" FOREIGN KEY (teacher_id) REFERENCES teachers(teacher_id)

Referenced by:
 TABLE "attendances" CONSTRAINT "attendances_session_id_fkey" FOREIGN KEY (session_id) REFERENCES class_sessions(session_id)

Structure of the Attendances table, showing recorded_at defaulting to now().

Table "public.attendances"				
Column	Type	Collation	Nullable	Default
attendance_id	integer		not null	nextval('attendances_attendance_id_seq'::regclass)
session_id	integer		not null	
student_id	integer		not null	
status	character varying(2)			
remarks	text			
recorded_at	timestamp without time zone			now()

Indexes:
 "attendances_pkey" PRIMARY KEY, btree (attendance_id)

Foreign-key constraints:
 "attendances_session_id_fkey" FOREIGN KEY (session_id) REFERENCES class_sessions(session_id)
 "attendances_student_id_fkey" FOREIGN KEY (student_id) REFERENCES students(student_id)

INSERT and SELECT — CRUD: Create and Read

Inserting two course records (BCA and BIT) and reading them back with SELECT * FROM Courses;

```
project=# INSERT INTO Courses (course_code, course_name, duration_years, description) VALUES
('BCA', 'Bachelor in Computer Applications', 4, 'Undergraduate program in computer applications'),
('BIT', 'Bachelor in Information Technology', 4, 'Undergraduate program in IT');
INSERT 0 2
project=# SELECT * FROM Courses;
```

course_id	course_code	course_name	duration_years	description
1	BCA	Bachelor in Computer Applications	4	Undergraduate program in computer applications
2	BIT	Bachelor in Information Technology	4	Undergraduate program in IT

(2 rows)

Inserting two teacher records and verifying with *SELECT * FROM Teachers;*

```
project=# INSERT INTO Teachers (first_name, last_name, email, phone, department, hire_date) VALUES
('Anil', 'Kumar', 'anil.kumar@example.com', '9800011111', 'Computer Science', '2018-06-15'),
('Sita', 'Rai', 'sita.raai@example.com', '9800022222', 'Mathematics', '2019-03-10');
INSERT 0 2
project=# SELECT * FROM Teachers;
```

teacher_id	first_name	last_name	email	phone	department	hire_date
1	Anil	Kumar	anil.kumar@example.com	9800011111	Computer Science	2018-06-15
2	Sita	Rai	sita.raai@example.com	9800022222	Mathematics	2019-03-10

(2 rows)

Inserting three student records and verifying with *SELECT * FROM Students;*

```
project=# INSERT INTO Students (course_id, first_name, last_name, email, phone, enrollment_year, symbol_number) VALUES
(1, 'Ram', 'Sharma', 'ram@example.com', '9800033333', 2023, 'SYM001'),
(1, 'Hari', 'Thapa', 'hari@example.com', '9800044444', 2023, 'SYM002'),
(2, 'Gita', 'Poudel', 'gita@example.com', '9800055555', 2023, 'SYM003');
INSERT 0 3
project=# SELECT * FROM Students;
```

student_id	course_id	first_name	last_name	email	phone	enrollment_year	symbol_number
1	1	Ram	Sharma	ram@example.com	9800033333	2023	SYM001
2	1	Hari	Thapa	hari@example.com	9800044444	2023	SYM002
3	2	Gita	Poudel	gita@example.com	9800055555	2023	SYM003

(3 rows)

Inserting two semester records and verifying with *SELECT * FROM Semesters;*

```
project=# INSERT INTO Semesters (course_id, semester_number, semester_name, start_date, end_date) VALUES
(1, 1, 'First Semester', '2023-08-01', '2023-12-31'),
(1, 2, 'Second Semester', '2024-01-01', '2024-05-31');
INSERT 0 2
project=# SELECT * FROM Semesters;
```

semester_id	course_id	semester_number	semester_name	start_date	end_date
1	1	1	First Semester	2023-08-01	2023-12-31
2	1	2	Second Semester	2024-01-01	2024-05-31

(2 rows)

Inserting two subject records and verifying with *SELECT * FROM Subjects;*

```
project=# INSERT INTO Subjects (semester_id, subject_code, subject_name, credit_hours, total_classes) VALUES
(1, 'CSC101', 'Introduction to Programming', 3, 45),
(1, 'MTH101', 'Basic Mathematics', 3, 45);
INSERT 0 2
project=# SELECT * FROM Subjects;
```

subject_id	semester_id	subject_code	subject_name	credit_hours	total_classes
1	1	CSC101	Introduction to Programming	3	45
2	1	MTH101	Basic Mathematics	3	45

(2 rows)

Assigning teachers to subjects through *Teacher_Subjects* and verifying the records.

```
project=# INSERT INTO Teacher_Subjects (teacher_id, subject_id, academic_year, assign_date) VALUES
(1, 1, '2023', '2023-07-20'),
(2, 2, '2023', '2023-07-20');
INSERT 0 2
project=# SELECT * FROM Teacher_Subjects;
```

ts_id	teacher_id	subject_id	academic_year	assign_date
1	1	1	2023	2023-07-20
2	2	2	2023	2023-07-20

(2 rows)

Enrolling three students into a semester through Enrollments and verifying the records.

```
project=# INSERT INTO Enrollments (student_id, semester_id, enrollment_date, status) VALUES
(1, 1, '2023-08-01', 'Active'),
(2, 1, '2023-08-01', 'Active'),
(3, 1, '2023-08-01', 'Active');
INSERT 0 3
project=# SELECT * FROM Enrollments;
```

enrollment_id	student_id	semester_id	enrollment_date	status
1	1	1	2023-08-01	Active
2	2	1	2023-08-01	Active
3	3	1	2023-08-01	Active

(3 rows)

Scheduling two class sessions through Class_Sessions and verifying the records.

```
project=# INSERT INTO Class_Sessions (subject_id, teacher_id, session_date, start_time, end_time, remarks) VALUES
(1, 1, '2023-08-10', '09:00', '10:30', 'First class of the semester'),
(2, 2, '2023-08-10', '11:00', '12:30', 'Introductory lecture');
INSERT 0 2
project=# SELECT * FROM Class_Sessions;
```

session_id	subject_id	teacher_id	session_date	start_time	end_time	remarks
1	1	1	2023-08-10	09:00:00	10:30:00	First class of the semester
2	2	2	2023-08-10	11:00:00	12:30:00	Introductory lecture

(2 rows)

Recording three attendance entries through Attendances and verifying the records, including the auto-generated recorded_at timestamp.

```
project=# INSERT INTO Attendances (session_id, student_id, status, remarks) VALUES
(1, 1, 'P', 'On time'),
(1, 2, 'A', 'Absent - no reason given'),
(2, 1, 'P', 'On time');
INSERT 0 3
project=# SELECT * FROM Attendances;
```

attendance_id	session_id	student_id	status	remarks	recorded_at
1	1	1	P	On time	2026-07-14 13:06:18.92083
2	1	2	A	Absent - no reason given	2026-07-14 13:06:18.92083
3	2	1	P	On time	2026-07-14 13:06:18.92083

(3 rows)

UPDATE — CRUD: Update

Updating a student's phone number using student_id and verified by SELECT.

```
project=# UPDATE Students
SET phone = '9811111111'
WHERE student_id = 1;
UPDATE 1
project=# SELECT * FROM Students;
```

student_id	course_id	first_name	last_name	email	phone	enrollment_year	symbol_number
2	1	Hari	Thapa	hari@example.com	9800044444	2023	SYM002
3	2	Gita	Poudel	gita@example.com	9800055555	2023	SYM003
1	1	Ram	Sharma	ram@example.com	9811111111	2023	SYM001

(3 rows)

Updating a teacher's department using teacher_id and verified by SELECT.

```
project=# UPDATE Teachers
SET department = 'Information Technology'
WHERE teacher_id = 2;
UPDATE 1
```

```
project=# SELECT * FROM Teachers;
```

teacher_id	first_name	last_name	email	phone	department	hire_date
1	Anil	Kumar	anil.kumar@example.com	9800011111	Computer Science	2018-06-15
2	Sita	Rai	sita.rao@example.com	9800022222	Information Technology	2019-03-10

(2 rows)

Updating an attendance record's status and remarks using attendance_id, verified by SELECT.

```
project=# UPDATE Attendances
SET status = 'P', remarks = 'Marked present after correction'
WHERE attendance_id = 2;
UPDATE 1
```

```
project=# SELECT * FROM Attendances;
```

attendance_id	session_id	student_id	status	remarks	recorded_at
1	1	1	P	On time	2026-07-14 13:06:18.92083
3	2	1	P	On time	2026-07-14 13:06:18.92083
2	1	2	P	Marked present after correction	2026-07-14 13:06:18.92083

(3 rows)

DELETE — CRUD: Delete

Deleting an attendance record using attendance_id, verified by SELECT.

```
project=# DELETE FROM Attendances
WHERE attendance_id = 2;
DELETE 1
```

```
project=# SELECT * FROM Attendances;
```

attendance_id	session_id	student_id	status	remarks	recorded_at
1	1	1	P	On time	2026-07-14 13:06:18.92083
3	2	1	P	On time	2026-07-14 13:06:18.92083

(2 rows)

Deleting an enrollment record using enrollment_id, verified by SELECT.

```
project=# DELETE FROM Enrollments
WHERE enrollment_id = 3;
DELETE 1
```

```
project=# SELECT * FROM Enrollments;
```

enrollment_id	student_id	semester_id	enrollment_date	status
1	1	1	2023-08-01	Active
2	2	1	2023-08-01	Active

(2 rows)

Deleting a student record using a combination of course_id and symbol_number, verified by SELECT.

```
project=# DELETE FROM Students
WHERE course_id = 2 AND symbol_number = 'SYM003';
DELETE 1
```

```
project=# SELECT * FROM Students;
```

student_id	course_id	first_name	last_name	email	phone	enrollment_year	symbol_number
2	1	Hari	Thapa	hari@example.com	9800044444	2023	SYM002
1	1	Ram	Sharma	ram@example.com	9811111111	2023	SYM001

(2 rows)

ALTER TABLE — Structure Modification (DDL)

Creating a temporary Library_Books table to demonstrate ALTER TABLE operations

```
project=# CREATE TABLE Library_Books (  
  book_id      SERIAL PRIMARY KEY,  
  title        VARCHAR(150),  
  author       VARCHAR(100),  
  isbn         VARCHAR(20),  
  copies       INT  
);  
CREATE TABLE
```

Verifying the initial structure of Library_Books with \d.

```
project=# \d Library_Books
```

Table "public.library_books"				
Column	Type	Collation	Nullable	Default
book_id	integer		not null	nextval('library_books_book_id_seq'::regclass)
title	character varying(150)			
author	character varying(100)			
isbn	character varying(20)			
copies	integer			

Indexes:
"library_books_pkey" PRIMARY KEY, btree (book_id)

Inserting two book records and verifying with SELECT * FROM Library_Books;

```
project=# INSERT INTO Library_Books (title, author, isbn, copies) VALUES  
( 'Database Systems', 'John Doe', '978-0-123456-01', 5),  
( 'Learning SQL', 'Jane Smith', '978-0-123456-02', 3);  
INSERT 0 2  
project=# SELECT * FROM Library_Books;
```

book_id	title	author	isbn	copies
1	Database Systems	John Doe	978-0-123456-01	5
2	Learning SQL	Jane Smith	978-0-123456-02	3

(2 rows)

Adding a new column, published_year, confirmed by \d.

```
project=# ALTER TABLE Library_Books  
ADD COLUMN published_year INT;  
ALTER TABLE  
project=# \d Library_Books
```

Table "public.library_books"				
Column	Type	Collation	Nullable	Default
book_id	integer		not null	nextval('library_books_book_id_seq'::regclass)
title	character varying(150)			
author	character varying(100)			
isbn	character varying(20)			
copies	integer			
published_year	integer			

Indexes:
"library books pkey" PRIMARY KEY, btree (book id)

Renaming the table using ALTER TABLE Library_Books RENAME TO Books;

```
project=# ALTER TABLE Library_Books
RENAME TO Books;
ALTER TABLE
project=#
```

Dropping the table with DROP TABLE Books; — \d Books then confirms the relation no longer exists.

```
project=# DROP TABLE Books;
DROP TABLE
project=# \d Books
Did not find any relation named "Books".
project=#
```

Advanced SELECT Queries — Multi-table JOINS

A multi-table JOIN linking Teacher_Subjects, Teachers, Subjects, Semesters and Courses to list every teacher's subject assignment.

```
project=# SELECT
    t.teacher_id,
    t.first_name || ' ' || t.last_name AS teacher_name,
    c.course_name,
    sem.semester_number,
    sem.semester_name,
    s.subject_code,
    s.subject_name,
    ts.academic_year,
    ts.assign_date
FROM teacher_subjects ts
JOIN teachers t
    ON ts.teacher_id = t.teacher_id
JOIN subjects s
    ON ts.subject_id = s.subject_id
JOIN semesters sem
    ON s.semester_id = sem.semester_id
JOIN courses c
    ON sem.course_id = c.course_id;
project=#
```

Result of the teacher–subject–course JOIN query.

teacher_id	teacher_name	course_name	semester_number	semester_name	subject_code	subject_name	academic_year	assign_date
1	Anil Kumar	Bachelor in Computer Applications	1	First Semester	CSC101	Introduction to Programming	2023	2023-07-20
2	Sita Rai	Bachelor in Computer Applications	1	First Semester	MTH101	Basic Mathematics	2023	2023-07-20

(2 rows)

A JOIN query and result showing each enrolled student's course, semester, and enrollment date.

```
project=# SELECT
  st.student_id,
  st.first_name || ' ' || st.last_name AS student_name,
  c.course_name,
  sem.semester_name,
  e.enrollment_date
FROM enrollments e
JOIN students st
  ON e.student_id = st.student_id
JOIN semesters sem
  ON e.semester_id = sem.semester_id
JOIN courses c
  ON sem.course_id = c.course_id;
 student_id | student_name |          course_name          | semester_name | enrollment_date
-----+-----+-----+-----+-----
          1 | Ram Sharma   | Bachelor in Computer Applications | First Semester | 2023-08-01
          2 | Hari Thapa   | Bachelor in Computer Applications | First Semester | 2023-08-01
(2 rows)
```

An explicit INNER JOIN between Teachers and Teacher_Subjects.

```
project=# SELECT *
FROM teachers t
INNER JOIN teacher_subjects ts
  ON t.teacher_id = ts.teacher_id;
project=#
```

Result of the INNER JOIN between Teachers and Teacher_Subjects.

teacher_id	first_name	last_name	email	phone	department	hire_date	ts_id	teacher_id	subject_id	academic_year	assign_date
1	Anil	Kumar	anil.kumar@example.com	9800011111	Computer Science	2018-06-15	1	1	1	2023	2023-07-20
2	Sita	Rai	sita.rao@example.com	9800022222	Information Technology	2019-03-10	2	2	2	2023	2023-07-20

(2 rows)

A LEFT JOIN between Courses and Semesters.

```
project=# SELECT *
FROM courses c
LEFT JOIN semesters s
  ON c.course_id = s.course_id;
project=#
```

Result of the LEFT JOIN — every course is returned even where there is no matching semester (e.g. BIT, which has none yet).

course_id	course_code	course_name	duration_years	description	semester_id	course_id	semester_number	semester_name	start_date	end_date
1	BCA	Bachelor in Computer Applications	4	Undergraduate program in computer applications	1	1	1	First Semester	2023-08-01	2023-12-31
1	BCA	Bachelor in Computer Applications	4	Undergraduate program in computer applications	2	1	2	Second Semester	2024-01-01	2024-05-31
2	BIT	Bachelor in Information Technology	4	Undergraduate program in IT						

(3 rows)

A RIGHT JOIN between Students and Enrollments

```
project=# SELECT *
FROM students st
RIGHT JOIN enrollments e
  ON st.student_id = e.student_id;
project=#
```

Result of the RIGHT JOIN between Students and Enrollments.

student_id	course_id	first_name	last_name	email	phone	enrollment_year	symbol_number	enrollment_id	student_id	semester_id	enrollment_date	status
1	1	Ram	Sharma	ram@example.com	9811111111	2023	SYM001	1	1	1	2023-08-01	Active
2	1	Hari	Thapa	hari@example.com	9800044444	2023	SYM002	2	2	1	2023-08-01	Active

(2 rows)

A comprehensive multi-table JOIN producing a full attendance report across students, courses, semesters, subjects, teachers and sessions

```
project=# SELECT
  st.student_id,
  st.first_name || ' ' || st.last_name AS student_name,
  c.course_name,
  sem.semester_name,
  s.subject_name,
  t.first_name || ' ' || t.last_name AS teacher_name,
  cs.session_date,
  a.status
FROM attendances a
JOIN students st
  ON a.student_id = st.student_id
JOIN class_sessions cs
  ON a.session_id = cs.session_id
JOIN subjects s
  ON cs.subject_id = s.subject_id
JOIN teachers t
  ON cs.teacher_id = t.teacher_id
JOIN semesters sem
  ON s.semester_id = sem.semester_id
JOIN courses c
  ON sem.course_id = c.course_id
JOIN enrollments e
  ON e.student_id = st.student_id
  AND e.semester_id = sem.semester_id
ORDER BY st.student_id, cs.session_date;
```

Result of the full attendance report query

student_id	student_name	course_name	semester_name	subject_name	teacher_name	session_date	status
1	Ram Sharma	Bachelor in Computer Applications	First Semester	Basic Mathematics	Sita Rai	2023-08-10	P
1	Ram Sharma	Bachelor in Computer Applications	First Semester	Introduction to Programming	Anil Kumar	2023-08-10	P

(2 rows)